

# Análisis de Resultados de Valgrind

Laboratorio 1 – Suma vectorial con pthreads

---

## Salida obtenida

```
$ valgrind --leak-check=full ./Laboratorio_1
==5204== Memcheck, a memory error detector
==5204== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==5204== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==5204== Command: ./Laboratorio_1
==5204==
==5204== Warning: set address range perms: large range [0x4a77040, 0x34567840) (undefined)
El resultado de la suma es: 0
estado_hilos[0] = 1
estado_hilos[1] = 1
estado_hilos[2] = 1
estado_hilos[3] = 1
==5204== Warning: set address range perms: large range [0x4a77028, 0x34567858) (noaccess)
==5204==
==5204== HEAP SUMMARY:
==5204==     in use at exit: 0 bytes in 0 blocks
==5204==   total heap usage: 6 allocs, 6 frees, 800,002,112 bytes allocated
==5204==
==5204== All heap blocks were freed -- no leaks are possible
==5204==
==5204== For lists of detected and suppressed errors, rerun with: -s
==5204== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

## 1. Advertencias de rango de memoria grande

Las líneas Warning: set address range perms: large range no son errores del programa. Aparecen porque el malloc del vector reserva un bloque grande (~800 MB, ya que  $N = 100,000,000$  elementos double), y Valgrind simplemente avisa que tuvo que marcar internamente ese rango grande como 'sin inicializar' y luego como 'sin acceso' al liberarlo.

## 2. Resultado de la suma: 0

Este resultado corresponde a la versión donde `acumulador_global` se declaró como `volatile char` en vez de `double`. Un char solo puede representar valores en el rango de -128 a 127 (con signo). Al sumar 100,000,000 veces el valor 1.0, el acumulador se desborda (overflow) constantemente y su valor 'da la vuelta' (wraparound) una y otra vez a lo largo de la ejecución.

Que el resultado final haya dado exactamente 0 es, en este caso, producto de dos factores combinados: el overflow del char y la forma en que los 4 hilos intercalaron sus escrituras sobre la misma variable (condición de carrera). No hay ninguna garantía de que el resultado sea siempre 0: al no existir un mutex protegiendo `acumulador_global`, cada corrida puede dar un valor distinto, dependiendo del orden exacto en que el sistema operativo intercale la ejecución de los hilos. En corridas anteriores sin Valgrind, con la misma variable en char, se observaron valores como 117, 91 y -98, lo cual confirma que el valor final no es determinístico.

Cabe aclarar que, en C, el overflow de un tipo con signo (como char con signo) es técnicamente comportamiento indefinido según el estándar del lenguaje. En la práctica, en gcc sobre arquitecturas

comunes (x86/x86-64), este wraparound se comporta de forma consistente como una operación módulo 256, pero no es un comportamiento garantizado por el estándar.

### 3. Estado de los hilos

Los cuatro valores de `estado_hilos[0..3] = 1` confirman que los 4 hilos llegaron a ejecutar la línea que marca su finalización, es decir, ningún hilo quedó bloqueado ni terminó de forma anormal.

### 4. HEAP SUMMARY

`in use at exit: 0 bytes in 0 blocks` indica que, al terminar el programa, no quedó memoria reservada sin liberar. El total de 6 `allocs`, 6 `frees` corresponde al `malloc/free` del vector más las asignaciones internas que hace la librería `pthread` al crear y administrar los hilos; todas fueron liberadas correctamente.

### 5. ERROR SUMMARY

`0 errors from 0 contexts` confirma que Memcheck no detectó accesos inválidos a memoria, uso de memoria sin inicializar, ni ningún otro error de manejo de memoria.

## Conclusión

**Memoria:** el programa está limpio — no hay fugas ni errores de acceso detectados por Memcheck.

**Tipo de dato:** declarar el acumulador como `char` provoca overflow, ya que el rango de este tipo es demasiado pequeño para el valor final esperado (100,000,000). El resultado 0 no es la suma real, sino un valor arbitrario producto del wraparound.

**Concurrencia:** además del overflow, sigue existiendo una condición de carrera real, ya que ningún mutex protege los accesos concurrentes a `acumulador_global`. Esto significa que el valor final (overflow incluido) es distinto en cada ejecución. Para confirmar la carrera de forma explícita se recomienda usar Helgrind o DRD (`valgrind --tool=helgrind ./Laboratorio_1`), que reportarían directamente el data race sobre esa variable.